

Walmart is the biggest retail store in the United States. Just like others, they have been expanding their e-commerce part of the business. By the end of 2022, e-commerce represented a roaring \$80 billion in sales, which is 13% of total sales of Walmart. One of the main factors that affects their sales is public holidays, like the Super Bowl, Labour Day, Thanksgiving, and Christmas.

In this project, you have been tasked with creating a data pipeline for the analysis of supply and demand around the holidays, along with conducting a preliminary analysis of the data. You will be working with two data sources: grocery sales and complementary data. You have been provided with the `grocery_sales` table in `PostgreSQL` database with the following features:

`grocery_sales`

- `"index"` - unique ID of the row
- `"Store_ID"` - the store number
- `"Date"` - the week of sales
- `"Weekly_Sales"` - sales for the given store

Also, you have the `extra_data.parquet` file that contains complementary data:

`extra_data.parquet`

- `"IsHoliday"` - Whether the week contains a public holiday - 1 if yes, 0 if no.
- `"Temperature"` - Temperature on the day of sale
- `"Fuel_Price"` - Cost of fuel in the region
- `"CPI"` - Prevailing consumer price index
- `"Unemployment"` - The prevailing unemployment rate
- `"Markdown1"`, `"Markdown2"`, `"Markdown3"`, `"Markdown4"` - number of promotional markdowns
- `"Dept"` - Department Number in each store
- `"Size"` - size of the store
- `"Type"` - type of the store (depends on `Size` column)

You will need to merge those files and perform some data manipulations. The transformed DataFrame can then be stored as the `clean_data` variable containing the following columns:

- `"Store_ID"`
- `"Month"`

- "Dept"
- "IsHoliday"
- "Weekly_Sales"
- "CPI"
- " "Unemployment" "

After merging and cleaning the data, you will have to analyze monthly sales of Walmart and store the results of your analysis as the `agg_data` variable that should look like:

Month	Weekly_Sales
1.0	33174.178494
2.0	34333.326579
...	...

Finally, you should save the `clean_data` and `agg_data` as the csv files.

It is recommended to use `pandas` for this project.



Projects Data DataFrame as grocery_sales

```
-- Write your SQL query here
SELECT * FROM grocery_sales
```

index	Store_ID	Date	Dept	Weekly_Sales
0	0	1 2010-02-05T00:00:00.000		1
1	1	1 2010-02-05T00:00:00.000		26
2	2	1 2010-02-05T00:00:00.000		17
3	3	1 2010-02-05T00:00:00.000		45
4	4	1 2010-02-05T00:00:00.000		28
5	5	1 2010-02-05T00:00:00.000		79
6	6	1 2010-02-05T00:00:00.000		55
7	7	1 2010-02-05T00:00:00.000		5
8	8	1 2010-02-05T00:00:00.000		58
9	9	1 2010-02-05T00:00:00.000		7
10	10	1 2010-02-05T00:00:00.000		83
11	11	1 2010-02-05T00:00:00.000		9
12	12	1 2010-02-05T00:00:00.000		87
13	13	1 2010-02-05T00:00:00.000		34
14	14	1 2010-02-05T00:00:00.000		19

20,000 rows ⚠ truncated from 231,522 rows [↓](#)

```
import pandas as pd
import os

def extract(store_data, extra_data):
    extra_df = pd.read_parquet(extra_data)
    merged_df = store_data.merge(extra_df, on = "index")
    return merged_df

# Call the extract() function and store it as the "merged_df" variable
merged_df = extract(grocery_sales, "extra_data.parquet")
```

```
# Create the transform() function with one parameter: "raw_data"
```

```
def transform(raw_data):
```

```
    # .fillna is used to fill in null values
```

```
    # syntax == DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None, downcast=None)
```

```
    # using mean values to fill in the fields
```

```
    raw_data.fillna(
```

```
        {
```

```
            'CPI': raw_data['CPI'].mean(),
```

```
            'Weekly_Sales': raw_data['Weekly_Sales'].mean(),
```

```
            'Unemployment': raw_data['Unemployment'].mean(),
```

```
        }, inplace = True # replacing the current value
```

```
    )
```

```
    #Extracting the Month from the Date column
```

```
    raw_data["Date"] = pd.to_datetime(raw_data["Date"], format = "%Y-%m-%d") #formatting the date column
```

```
    raw_data["Month"] = raw_data["Date"].dt.month
```

```
    #Filtering the weekly sales over $10,000
```

```
    raw_data = raw_data.loc[raw_data["Weekly_Sales"] > 10000, :] #label based indexing, selecting all cols that satisfy the condition
```

```
    #Dropping the unwanted columns
```

```
    raw_data = raw_data.drop(["index", "Temperature", "Fuel_Price", "MarkDown1", "MarkDown2", "MarkDown3", "MarkDown4", "MarkDown5", "Type", "Size", "Date"], axis = 1) #axis=1 means we are dropping the cols
```

```
    return raw_data
```

```
# Call the transform() function and pass the merged DataFrame
```

```
clean_data = transform(merged_df)
```

```
# Create the avg_weekly_sales_per_month function that takes in the cleaned data from the last step
```

```
def avg_weekly_sales_per_month(clean_data):
```

```
    holiday_sales = clean_data[["Month", "Weekly_Sales"]]
```

```
#chain functions
holiday_sales = holiday_sales.groupby("Month")["Weekly_Sales"].mean().reset_index(name="Avg_Sales").round(2) #to have the
default index
return holiday_sales
```

```
# Call the avg_weekly_sales_per_month() function and pass the cleaned DataFrame
agg_data = avg_weekly_sales_per_month(clean_data)
```

```
# Create the load() function that takes in the cleaned DataFrame and the aggregated one with the paths where they are going to be
stored
def load(full_data, full_data_file_path, agg_data, agg_data_file_path):

    # Save both DataFrames as csv files. Set index = False to drop the index columns
    full_data.to_csv(full_data_file_path, index = False)
    agg_data.to_csv(agg_data_file_path, index = False)
```

```
# Call the load() function and pass the cleaned and aggregated DataFrames with their paths
load(clean_data, "clean_data.csv", agg_data, "agg_data.csv")
```